

Deployment Readiness Checklist

CIS Technologies — work management platform

Cloud-agnostic · everything to do before and after your first push to production

Read this first — two blockers will break a deploy today

1. There are no database migrations.

`apps/api/prisma/` contains only `schema.prisma`. `npm run db:migrate` runs `prisma migrate deploy`, which with no `migrations/` folder does nothing at all and exits 0. Deploy as-is and your production API starts against an empty database. Fixed in §1.1.

2. The deployed web app cannot reach the API.

Vite bakes `VITE_API_BASE_URL` in at build time; the web Dockerfile never sets it, so the bundle calls `/api/v1/...` on its own origin — and `nginx.conf` has no `/api` proxy. Every request 404s. Fixed in §1.2.

1 · Blockers to clear before anything else

1.1 Create the migration baseline blocker

Development used `prisma db push`, which mutates the database without recording history. Production needs versioned, reviewable migrations.

For a brand-new production database — generate the initial migration from the schema:

```
cd apps/api
npx prisma migrate dev --name init      # writes prisma/migrations/..._init/
git add prisma/migrations && git commit -m "Add initial migration"
```

If a database already exists with the schema in it (your local database is in this state), baseline it instead so Prisma does not try to recreate tables:

```
npx prisma migrate diff \
  --from-empty --to-schema-datamodel prisma/schema.prisma \
  --script > prisma/migrations/0_init/migration.sql

npx prisma migrate resolve --applied 0_init
```

Never run `prisma db push` against production.

It has no history, no review step and no rollback, and it will drop columns it believes are unused. Development only.

1.2 Decide how the browser reaches the API blocker

Pick one. Option A is recommended: one origin means no CORS, no build-time URL, and cookies stay first-party if you later move tokens into them.

Option	What to do	Trade-off
--------	------------	-----------

A. One domain, path routing recommended	Route <code>/api/*</code> to the API and everything else to the web container at your load balancer. Leave <code>VITE_API_BASE_URL</code> empty.	No CORS. Needs a load balancer that supports path rules.
B. nginx proxy	Add to <code>apps/web/nginx.conf</code> : <pre>location /api/ { proxy_pass http://api:4000; proxy_set_header Host \$host; proxy_set_header X-Forwarded-For \$proxy_add_x_forwarded_for; proxy_set_header X-Forwarded-Proto \$scheme; }</pre>	Web container must resolve the API by DNS.
C. Separate API domain	Add a build arg to <code>apps/web/Dockerfile</code> : <pre>ARG VITE_API_BASE_URL ENV VITE_API_BASE_URL=\$VITE_API_BASE_URL</pre> then build with <code>--build-arg VITE_API_BASE_URL=https://api.example.com</code> .	CORS applies. <code>WEB_BASE_URL</code> must match the web origin exactly.

Whichever option you choose:

the API's `WEB_BASE_URL` is the single allowed CORS origin (`apps/api/src/app.ts`). Get it wrong and the app loads but no data ever arrives.

2 · Before you push — code and repository

- ☐ **All three gates pass locally.** `npm run typecheck && npm run lint && npm run test`
- ☐ **Production build succeeds.** `npm run build` — not covered by the gates above.
- ☐ **Both images build.** `docker build -f apps/api/Dockerfile -t api` . and the same for `apps/web` .
- ☐ **package-lock.json is committed.** CI and both Dockerfiles use `npm ci` , which fails without it.
- ☐ **No .env file is committed.** Check with `git ls-files | grep -c "\.env$"` — it must print 0.
- ☐ **Every development secret is regenerated** for production (see §3).
- ☐ **Seeding is not wired into container start.** `npm run db:seed` must be a deliberate one-off, never part of the API boot path.
- ☐ **CI is green on the branch you are deploying**, including the Docker build job.

3 · Configuration and secrets

Validated at boot by `apps/api/src/config/env.ts` . A missing or malformed value exits the process before it serves a request — deliberate, but it means a typo presents as a crash loop.

Variable	Required	Production guidance
<code>NODE_ENV</code>	yes	<code>production</code> . Anything else keeps development behaviour.

DATABASE_URL	yes	Managed Postgres 16. Append <code>?sslmode=require</code> . Never shared with another app.
JWT_ACCESS_SECRET	yes	At least 32 chars, unique per environment.
JWT_REFRESH_SECRET	yes	Different value from the access secret. Rotating it signs everyone out.
WEB_BASE_URL	yes	Exact public web origin — this is the CORS allow-list.
API_BASE_URL	yes	Public API origin. Used to build links in outgoing email.
REDIS_URL	defaulted	Managed Redis 7. The default localhost value will fail in the cloud.
EMAIL_DRIVER	defaulted	Set to <code>smtp</code> . The default <code>console</code> silently prints invitations to the log instead of sending them.
SMTP_*, EMAIL_FROM	if smtp	Sender must be a domain verified with SPF, DKIM and DMARC.
VAPID_PUBLIC_KEY / _PRIVATE_KEY	optional	Generate fresh per environment. Absent means no browser push; the in-app bell still works.
CALENDAR_DRIVER	defaulted	<code>none</code> means meetings get no join link. <code>google</code> needs the three <code>GOOGLE_*</code> values.
GOOGLE_REDIRECT_URI	if google	Must exactly equal the production callback registered in Google Cloud.
SIGNUP_ORG_SLUG	optional	Only needed once a second organisation exists. Signup errors clearly rather than guessing.
RATE_LIMIT_MAX	defaulted	300/min/IP. Behind a proxy this counts the <i>proxy's</i> IP unless trust proxy is set — see §5.

```
# generate a full set of production secrets
openssl rand -base64 48 # JWT_ACCESS_SECRET
openssl rand -base64 48 # JWT_REFRESH_SECRET
node -e "console.log(require('web-push').generateVAPIDKeys())"
```

Do not reuse the development VAPID keys or JWT secrets.

They exist in shell history, in the working tree and in any screenshot shared during development. Treat every one of them as public.

4 · Database

- ☐ **A dedicated database.** Not shared with another application, and not a schema inside someone else's database.
- ☐ **Automated backups on, with a tested restore.** An untested backup is a hope, not a backup.
- ☐ **Point-in-time recovery enabled.** Decide the acceptable data-loss window and confirm retention actually meets it.
- ☐ **TLS enforced** on the connection (`sslmode=require` or stricter).
- ☐ **Least-privilege application user** — DML and DDL for migrations, but not superuser and not able to create databases.
- ☐ **Migration runs as its own step** before the new API rolls out, never on container start — two replicas booting at once would race each other.
- ☐ **Connection-pool sizing checked.** Prisma defaults to `num_cpus × 2 + 1` per instance; replicas × pool size must stay under the database's `max_connections`.

Seeding production

`prisma/seed.ts` creates a demo organisation, four users sharing one password, and sample boards. **Never run it against production.** Create exactly one real owner, then use the in-app invitation and approval flows.

5 · Security review before go-live

Area	What to verify
HTTPS everywhere	TLS at the load balancer, HTTP redirected, HSTS enabled.
Trust proxy	Express must trust the proxy or rate limiting and audit IPs all see the load balancer address. Set <code>app.set('trust proxy', 1)</code> .
CORS	<code>WEB_BASE_URL</code> is one exact origin. Never a wildcard.
Secrets at rest	Injected from a secret manager. Not in the image, not inline in a task definition, not in git.
Refresh-token storage	accepted risk The refresh token lives in <code>localStorage</code> (<code>useAuth.tsx</code>) — a deliberate, documented choice, but readable by any XSS. Move to an <code>httpOnly</code> cookie if you handle sensitive data.
Password policy	12+ chars with upper, lower and digit, enforced by one shared schema on client and server. Argon2id hashing.
Self-signup	Anyone can request an account. Confirm an admin watches the approvals queue, or disable the route before launch.
Role boundaries	Re-test after deploy: MANAGER sees all work but not Admin; MEMBER sees only their own boards, tasks and meetings.
Container user	Both images already run as non-root. Keep it that way.
Dependency audit	<code>npm audit --omit=dev</code> — triage anything high or critical.

6 · Release sequence

1. **Tag the release.** `git tag -a v1.0.0 -m "..." && git push --tags`
2. **Build and push both images** tagged with the commit SHA — never only `latest`, or you cannot roll back.
3. **Run migrations** as a one-off task on the new image: `npx prisma migrate deploy`. Wait for success.
4. **Deploy the API.** Wait for `/api/v1/health/ready` to return 200 — it checks database and cache.
5. **Deploy the web bundle.**
6. **Run the smoke tests in §7.**
7. **Watch 5xx rate and job failures** for one full release cycle before walking away.

Migrations must stay backwards compatible with the running code.

Add columns nullable, backfill, make them required in a *later* release. Rolling back means deploying the previous image — it does not mean reversing a migration.

7 · After deploy — verification runbook

```
curl -fsS https://YOUR_DOMAIN/api/v1/health/live      # process is up
curl -fsS https://YOUR_DOMAIN/api/v1/health/ready    # database + redis reachable
# expect: {"success":true,"data":{"status":"ready","checks":{"database":true,"cache":true}}}
```

- ☐ Web app loads over HTTPS with no console errors.
- ☐ Sign in as the owner account you created.
- ☐ Create a workspace, a board and a task; reload and confirm all three persisted.
- ☐ Change a task's status and owner; confirm the assignee gets a notification.
- ☐ Schedule a meeting; confirm the clash warning appears for an overlapping time.
- ☐ Request an account from the sign-in page; confirm it appears in Admin → Approvals and that signing in before approval is refused.
- ☐ Sign in as a MEMBER: the Admin nav is absent and `/admin` redirects home.
- ☐ Confirm a MEMBER cannot open a board they are not on, by pasting its URL.
- ☐ Trigger an invitation and confirm the email actually arrives — not merely that it queued.
- ☐ Export a board to CSV and open the file.

8 · Operating it

Monitor

- 5xx rate and p95 latency per route
- `/health/ready` as the load-balancer health check — `/health/live` only proves the process exists
- Database CPU, connection count, free storage
- Redis memory and evictions
- Email queue depth and failure count
- TLS certificate expiry

Alert on

- Ready check failing on any instance for more than 2 minutes
- 5xx rate above your error budget
- Database connections above roughly 80% of max
- Disk above 80%

Log hygiene

Logging is structured JSON via pino and request bodies are redacted. Confirm no secret or password reaches the log aggregator before shipping logs to a third party.

9 · Known gaps — decide before launch

Gap	Impact and options
No SSO / SAML / SCIM	Enterprise buyers will ask. Password auth only today. A multi-week piece of work.
No file attachments	Tasks take comments but not files. Needs an object-store decision first.
Custom fields are not board columns	Definable and storable through Admin, but they do not render on the board yet.
No scheduled digests	CSV export exists; the Monday-morning email does not. The queue is already in place.
Microsoft calendar unimplemented	<code>CALENDAR_DRIVER=microsoft</code> silently falls back to no links.
Email worker shares the API process	Fine at low volume. Split into its own replica set as volume grows.
Dashboard aggregate computed live	The first query likely to become slow. Move behind a materialised view when it does.

10 · One-page summary

Phase	Must be done
Blockers	Create migration baseline · choose web-to-API routing
Before	Gates green · images build · lockfile committed · no <code>.env</code> in git · fresh secrets · seed disarmed
Config	All required env set · <code>EMAIL_DRIVER=smtp</code> · <code>WEB_BASE_URL</code> exact · secrets from a manager
Data	Dedicated database · backups plus tested restore · TLS · migration as its own step
Deploy	SHA-tagged images · migrate · API · wait for ready · web
After	Health checks · smoke tests · role boundaries · real email delivery · monitoring live